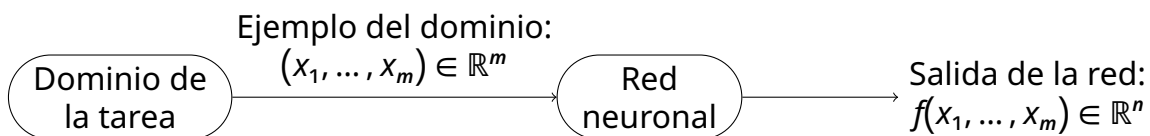


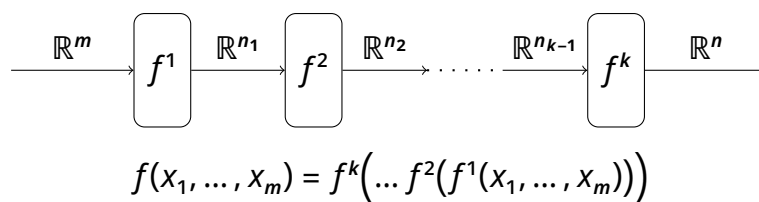
Redes neuronales

Una red neuronal con alimentación hacia adelante (*feedforward neural network*, en inglés), es un modelo de regresión para aprendizaje supervisado a partir de atributos numéricos. Como tal implementa una función $f: \mathbb{R}^m \rightarrow \mathbb{R}^n$, cuyos argumentos $x_1, \dots, x_m$ son atributos numéricos.

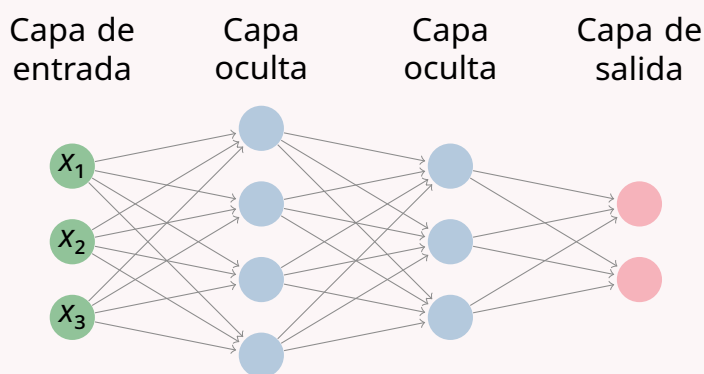


Las redes neuronales se inspiran en la estructura y funcionamiento del cerebro, aunque no son un modelo de este. Un cerebro humano se compone de un enorme número de neuronas (se estima que contiene unos ochenta mil millones, 8×10^{10} , de neuronas), aunque aún más importante que la cantidad es el hecho de que estas neuronas se interconectan entre sí. De esta forma, a través de estas interconexiones una neurona puede enviar información (en forma de señales eléctricas) a otras neuronas a las que esté conectada. Estas neuronas, a su vez, propagarán esa información a otras neuronas, y así sucesivamente.

De manera análoga, las redes neuronales se construyen a partir de unas unidades básicas, que llamaremos neuronas artificiales, organizadas en capas. De esta forma, la red neuronal calcula la función f como la composición de las funciones $f^i: \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$ calculadas en cada capa:



Es habitual representar las redes neuronales con alimentación hacia adelante como un grafo acíclico en el que los nodos son las neuronas artificiales y las aristas conectan todas las neuronas de cada capa con cada una de las neuronas de la capa siguiente. Suele incluirse una primera capa, de entrada, que no calcula nada, sino que simplemente proporciona a la siguiente capa los argumentos de la función. La última capa, de salida, es la que proporciona el resultado de la función. El resto de capas, entre la capa de entrada y la de salida, se llaman capas ocultas, ya que los resultados proporcionados por esas capas no son observables por un observador externo.

Ejemplo: Representación gráfica de una red neuronal

Esta red neuronal tiene una capa de entrada con tres neuronas, dos capas ocultas con cuatro y tres neuronas, respectivamente, y una capa de salida con dos neuronas:

- La primera capa oculta define una función $f^1: \mathbb{R}^3 \rightarrow \mathbb{R}^4$ dada por

$$f^1(x_1, x_2, x_3) = (a_1^1, a_2^1, a_3^1, a_4^1)$$

donde cada a_i^1 es la salida de la neurona i -ésima de la capa, calculada a partir de los argumentos x_1, x_2, x_3 .

- La segunda capa oculta define una función $f^2: \mathbb{R}^4 \rightarrow \mathbb{R}^3$ dada por

$$f^2(a_1^1, a_2^1, a_3^1, a_4^1) = (a_1^2, a_2^2, a_3^2)$$

donde cada a_i^2 es la salida de la neurona i -ésima de la capa, calculada a partir de los argumentos $a_1^1, a_2^1, a_3^1, a_4^1$.

- La capa de salida define una función $f^3: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ dada por

$$f^3(a_1^2, a_2^2, a_3^2) = (a_1^3, a_2^3)$$

donde cada a_i^3 es la salida de la neurona i -ésima de la capa, calculada a partir de los argumentos a_1^2, a_2^2, a_3^2 .

La red neuronal define, por tanto, una función $f: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ dada por

$$f(x_1, x_2, x_3) = f^3(f^2(f^1(x_1, x_2, x_3)))$$

Perceptrón

Un perceptrón es un tipo de neurona artificial que para cada entrada proporciona como salida 0 o 1. Puede usarse como modelo de clasificación binaria, identificando por ejemplo la salida 1 con la clase positiva y la salida 0 con la clase negativa.

Dados los argumentos $x_1, \dots, x_m$, la salida del perceptrón se calcula como sigue:

$$\begin{aligned} a &= \begin{cases} 1 & \text{si } \sum_{i=1}^m w_i x_i + b > 0, \\ 0 & \text{si } \sum_{i=1}^m w_i x_i + b \leq 0. \end{cases} \\ &= \begin{cases} 1 & \text{si } \mathbf{w}^T \mathbf{x} + b > 0, \\ 0 & \text{si } \mathbf{w}^T \mathbf{x} + b \leq 0. \end{cases} \\ &= \text{umbral}(z) \end{aligned}$$

donde $\mathbf{x}$ es el vector de los argumentos, $\mathbf{w}$ es el vector de pesos asociados a los argumentos, b es el sesgo de activación, $z = \mathbf{w}^T \mathbf{x} + b$ es la entrada del perceptrón y *umbral* es la función de activación, definida como

$$\text{umbral}(z) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{si } z > 0, \\ 0 & \text{si } z \leq 0. \end{cases}$$

En el campo del álgebra matricial, los vectores se consideran como matrices columna. Por economía de espacio, en el documento escribimos los vectores como $\mathbf{v} = (v_1 \dots v_m)^T$ en lugar de $\mathbf{v} = \begin{pmatrix} v_1 \\ \vdots \\ v_m \end{pmatrix}$, donde T denota la operación de trasposición.

De esta forma,

$$\mathbf{x} = (x_1 \dots x_m)^T \quad \mathbf{w} = (w_1 \dots w_m)^T \quad \mathbf{w}^T \mathbf{x} = (w_1 \dots w_m) \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix} = \sum_{i=1}^m w_i x_i$$

También es posible usar como función de activación la función *signo*:

$$\text{signo}(z) \stackrel{\text{def}}{=} \begin{cases} 1 & \text{si } z > 0, \\ -1 & \text{si } z \leq 0. \end{cases}$$

en cuyo caso el perceptrón para cada entrada proporcionará como salida -1 o 1 .

Ejemplo: Cálculo de la función lógica NOT mediante un perceptrón

El perceptrón con argumento x_1 , peso asociado al argumento $w_1 = -1$ y sesgo $b = 1$ calcula la función lógica NOT.

En efecto:

- Para $x_1 = 0$ se tiene que $z = w_1 x_1 + b = 1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$.
- Para $x_1 = 1$ se tiene que $z = w_1 x_1 + b = 0$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$.

Ejemplo: Cálculo de la función lógica AND mediante un perceptrón

El perceptrón con argumentos x_1 y x_2 , pesos asociados a los argumentos $w_1 = 1$ y $w_2 = 1$ y sesgo $b = -1$ calcula la función lógica AND.

En efecto:

- Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_1x_1 + w_2x_2 + b = -1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$.
- Para $x_1 = 0$ y $x_2 = 1$ se tiene que $z = w_1x_1 + w_2x_2 + b = 0$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$.
- Para $x_1 = 1$ y $x_2 = 0$ se tiene que $z = w_1x_1 + w_2x_2 + b = 0$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$.
- Para $x_1 = 1$ y $x_2 = 1$ se tiene que $z = w_1x_1 + w_2x_2 + b = 1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$.

El sesgo b establece con qué facilidad se activa el perceptrón. A mayor valor de b , mayor valor de z y, en consecuencia, más fácil es obtener $a = 1$ como salida. Por el contrario, a menor valor de b , menor valor de z y, en consecuencia, más difícil es obtener $a = 1$ como salida.

La inclusión del sesgo facilita el entrenamiento del perceptrón. En efecto, puesto que para clasificar un ejemplo $(x_1, \dots, x_n)$ calcula una combinación lineal de esos argumentos, el resultado final es que el perceptrón establece el hiperplano definido por la ecuación $\sum_{i=1}^m w_i x_i + b = 0$ como frontera entre los ejemplos positivos y los negativos. Si no se incluyera el sesgo, esa frontera siempre incluiría el punto $(0, \dots, 0)$ y se dificultaría la tarea de clasificación correspondiente.

Para simplificar la notación, es habitual considerar un argumento virtual $x_0 = 1$ con peso asociado $w_0 = b$. De esta forma, podemos escribir $z = \sum_{i=0}^m w_i x_i$. La función definida por el perceptrón es, entonces, $f = f(\mathbf{x}; \mathbf{w})$, donde los pesos $w_0, \dots, w_m$ son los parámetros que debe aprender el modelo.

Un perceptrón se entrena mediante aprendizaje supervisado. Dado un conjunto de ejemplos de entrenamiento, para cada uno de ellos se compara la salida pretendida con la salida proporcionada por el perceptrón, actualizando el valor de los pesos en el sentido que disminuya el error cometido.

- 1 Inicializar los pesos aleatoriamente
- 2 **Repetir** hasta que se cumpla la condición de terminación
- 3 **Para cada** ejemplo de entrenamiento
- 4 **Si** la salida del perceptrón es incorrecta **entonces**
- 5 Actualizar los pesos
- 6 **Devolver** los pesos

Si $(x_1, \dots, x_m, y)$ es un ejemplo de entrenamiento con salida a incorrecta, entonces los pesos se actualizan así:

$$w_i \leftarrow w_i + \eta(y - a)x_i \quad \text{para cada } i = 0, \dots, m$$

donde el hiperparámetro $\eta > 0$ es el *factor de aprendizaje*.

Un perceptrón solo es capaz de clasificar correctamente un conjunto de ejemplos si este es linealmente separable (es decir, los ejemplos positivos están separados de los negativos por un hiperplano). Por un lado, la condición es necesaria, ya que el perceptrón establece un hiperplano como frontera de clasificación. Por otro lado, el siguiente resultado establece que la condición es suficiente.

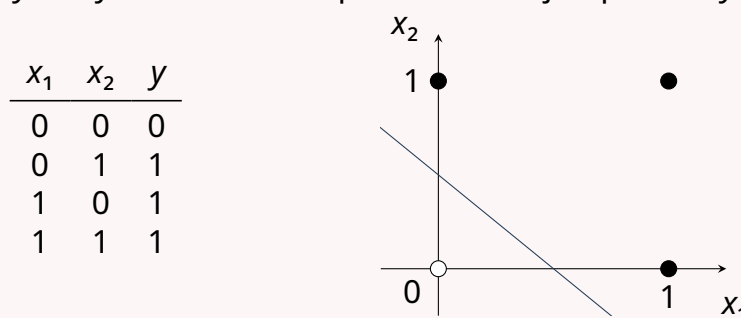
Teorema (Minsky y Papert, 1969)

Si $\mathcal{D}$ es un conjunto de ejemplos linealmente separable y η es un factor de aprendizaje suficientemente pequeño, entonces el algoritmo de entrenamiento del perceptrón converge en un número finito de pasos a un vector de pesos tal que todos los ejemplos de $\mathcal{D}$ se clasifican correctamente.

Por lo tanto, si se cumplen las condiciones del teorema se puede establecer como condición de terminación del algoritmo de entrenamiento del perceptrón el que todos los ejemplos se clasifiquen correctamente.

Ejemplo: Entrenamiento de un perceptrón para calcular la función lógica OR

El conjunto de entrenamiento es linealmente separable (el símbolo $\circ$ representa un ejemplo con $y = 0$ y el símbolo $\bullet$ representa un ejemplo con $y = 1$):



Supongamos $w_0 = 0.1$, $w_1 = -0.1$ y $w_2 = 0.1$ como pesos iniciales y $\eta = 0.1$ como factor de aprendizaje.

- Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es incorrecta.

Los pesos se actualizan como sigue:

$$w_0 \leftarrow w_0 + \eta(0 - 1)1 = 0.1 - 0.1 = 0$$

$$w_1 \leftarrow w_1 + \eta(0 - 1)x_1 = -0.1 - 0 = -0.1$$

$$w_2 \leftarrow w_2 + \eta(0 - 1)x_2 = 0.1 - 0 = 0.1$$

- Para $x_1 = 0$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.
- Para $x_1 = 1$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = -0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$, que es incorrecta.

Los pesos se actualizan como sigue:

$$w_0 \leftarrow w_0 + \eta(1 - 0)1 = 0 + 0.1 = 0.1$$

$$w_1 \leftarrow w_1 + \eta(1 - 0)x_1 = -0.1 + 0.1 = 0$$

$$w_2 \leftarrow w_2 + \eta(1 - 0)x_2 = 0.1 + 0 = 0.1$$

- Para $x_1 = 1$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.2$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.
- Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es incorrecta.

Los pesos se actualizan como sigue:

$$w_0 \leftarrow w_0 + \eta(0 - 1)1 = 0.1 - 0.1 = 0$$

$$w_1 \leftarrow w_1 + \eta(0 - 1)x_1 = 0 - 0 = 0$$

$$w_2 \leftarrow w_2 + \eta(0 - 1)x_2 = 0.1 - 0 = 0.1$$

- Para $x_1 = 0$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.
- Para $x_1 = 1$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$, que es incorrecta.

Los pesos se actualizan como sigue:

$$w_0 \leftarrow w_0 + \eta(1 - 0)1 = 0 + 0.1 = 0.1$$

$$w_1 \leftarrow w_1 + \eta(1 - 0)x_1 = 0 + 0.1 = 0.1$$

$$w_2 \leftarrow w_2 + \eta(1 - 0)x_2 = 0.1 + 0 = 0.1$$

- Para $x_1 = 1$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.3$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.
- Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es incorrecta.

Los pesos se actualizan como sigue:

$$w_0 \leftarrow w_0 + \eta(0 - 1)1 = 0.1 - 0.1 = 0$$

$$w_1 \leftarrow w_1 + \eta(0 - 1)x_1 = 0.1 - 0 = 0.1$$

$$w_2 \leftarrow w_2 + \eta(0 - 1)x_2 = 0.1 - 0 = 0.1$$

- Para $x_1 = 0$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.
- Para $x_1 = 1$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.

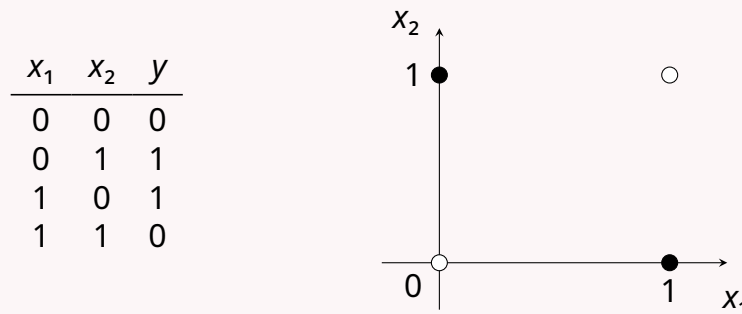
- Para $x_1 = 1$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.2$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 1$, que es correcta.
- Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0$, por lo que la salida del perceptrón es $a = \text{umbral}(z) = 0$, que es correcta.

Por lo tanto, el algoritmo ha entrenado un perceptrón que tiene $w_0 = 0$, $w_1 = 0.1$ y $w_2 = 0.1$ como pesos de los argumentos y que clasifica correctamente todos los ejemplos del conjunto de entrenamiento.

Un ejemplo clásico de conjunto de ejemplos no linealmente separable es el que se tiene para la función lógica XOR (disyunción exclusiva). Un perceptrón no puede, en consecuencia, aprender correctamente este conjunto de ejemplos en su totalidad.

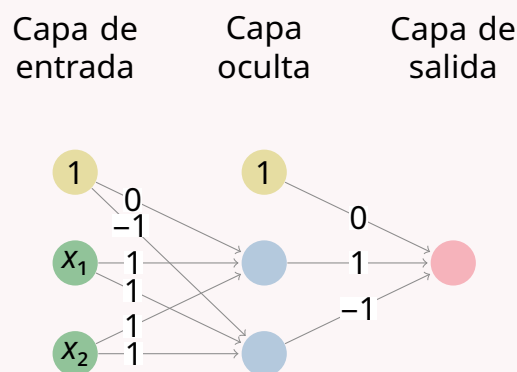
Ejemplo: No separabilidad lineal de la función XOR

El conjunto de entrenamiento no es linealmente separable (el símbolo $\circ$ representa un ejemplo con $y = 0$ y el símbolo $\bullet$ representa un ejemplo con $y = 1$):



Queda patente, pues, que los perceptrones tienen una capacidad expresiva limitada, lo que lleva a la necesidad de construir redes multicapas de perceptrones que, mediante la combinación de más de una de esas unidades, proporcionen mayor capacidad expresiva.

Ejemplo: Cálculo de la función lógica XOR mediante una red multicapa de perceptrones



- La entrada del primer perceptrón de la capa oculta es $z = x_1 + x_2$, luego solo se activa cuando alguno de los dos argumentos, o ambos, tiene el valor 1. La salida de este perceptrón es $a_1^1 = 1$ en ese caso y $a_1^1 = 0$ en caso contrario.
- La entrada del segundo perceptrón de la capa oculta es $z = x_1 + x_2 - 1$, luego solo se activa cuando ambos argumentos tienen el valor 1. La salida de este perceptrón es $a_2^1 = 1$ en ese caso y $a_2^1 = 0$ en caso contrario.
- La entrada del perceptrón de la capa de salida es $z = a_1^1 - a_2^1$, luego solo se activa cuando se ha activado el primer perceptrón de la capa oculta, pero no el segundo. Es decir, cuando alguno de los dos argumentos, pero no ambos, tiene el valor 1.

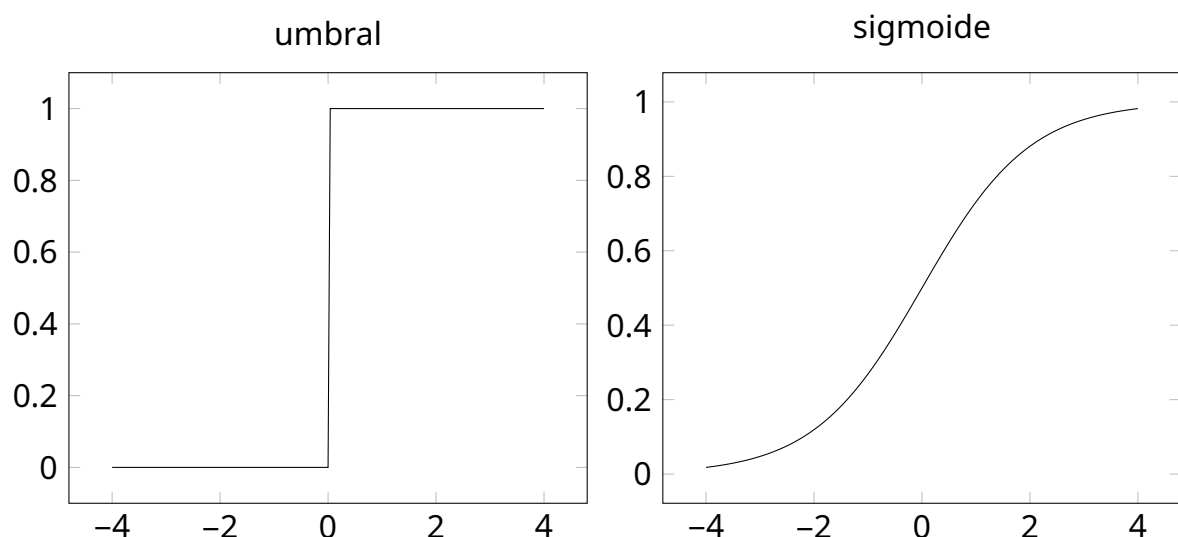
Método del descenso por el gradiente

Para poder concebir un algoritmo de entrenamiento para las redes neuronales, nos interesa que un cambio pequeño en los pesos produzca un cambio pequeño en la salida de la función implementada por la red. Esto no ocurre si la red está formada por perceptrones. Una manera de superar este problema es cambiar la función de activación por una función que pase del valor 0 al valor 1 de una forma más suave que la función *umbral* (y, como veremos más adelante, también interesa que sea diferenciable). Se tiene entonces lo que conocemos propiamente como neurona artificial.

Una opción habitual es usar como función de activación la función sigmoide

$$\sigma(z) \stackrel{\text{def}}{=} \frac{1}{1 + e^{-z}}$$

cuya gráfica podemos comparar con la de la función *umbral*:



Esta función tiene la propiedad de que $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

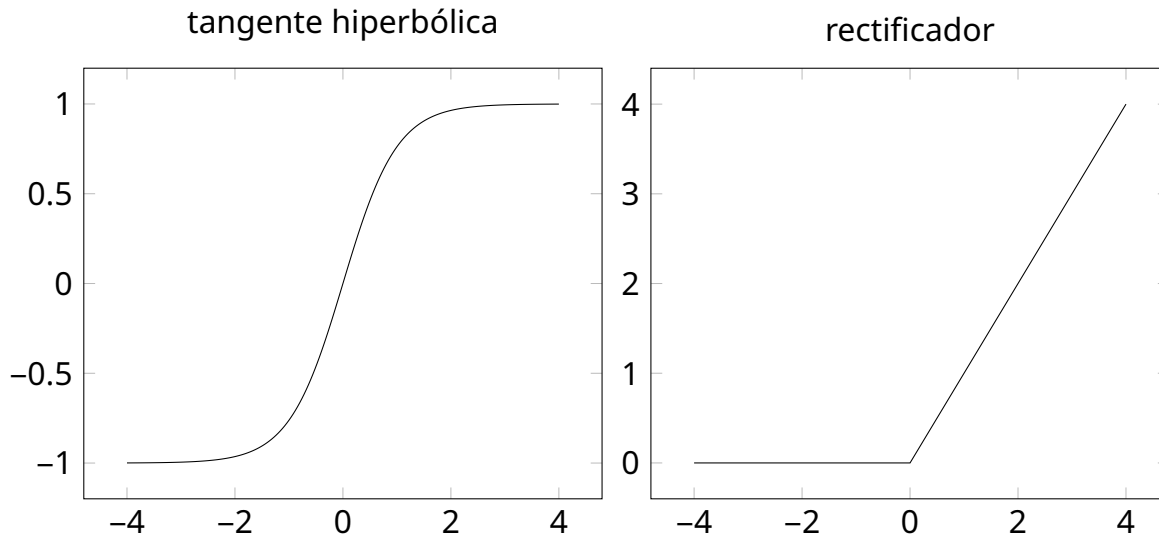
Otras funciones de activación que también se suelen utilizar son la función tangente hiperbólica o la función rectificador (una neurona artificial que use esta última como función de activación se conoce como unidad lineal rectificada, ReLU):

$$\tanh(z) \stackrel{\text{def}}{=} \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - \tanh(z)^2$$

$$\text{rectificador}(z) \stackrel{\text{def}}{=} \max(0, z)$$

$$\text{rectificador}'(z) = \begin{cases} 0 & \text{si } z \leq 0, \\ 1 & \text{si } z > 0. \end{cases}$$



En realidad, la función rectificador no es derivable en $z = 0$, pero en una implementación práctica nunca se va a necesitar para ese valor, por lo que la derivada se puede definir con un valor arbitrario, en este caso el 0.

De hecho, aunque las primeras funciones de activación que se estudiaron fueron la sigmoide y la tangente hiperbólica, en la actualidad se recomienda utilizar ReLUs en las capas ocultas de las redes neuronales.

Una neurona artificial usando como función de activación cualquiera de estas funciones se convierte en un modelo de regresión, ya que la salida se transforma en una variable continua con valores en un cierto intervalo ($(0, 1)$ para la sigmoide, $(-1, 1)$ para la tangente hiperbólica y $[0, +\infty)$ para la función rectificador). Para volver a convertir a la neurona artificial en un modelo de clasificación binaria basta discretizar la salida fijando un determinado umbral u : los ejemplos para los que la salida es mayor que u se clasifican como positivos y los ejemplos para los que la salida es menor o igual que u como negativos. No obstante, la neurona artificial seguiría separando los ejemplos positivos de los negativos mediante un hiperplano. Por ejemplo, para la función de activación sigmoide,

$$\sigma(\mathbf{w}^T \mathbf{x}) > u \Leftrightarrow \mathbf{w}^T \mathbf{x} > \sigma^{-1}(u)$$

El entrenamiento de una neurona artificial, como modelo de regresión, se realiza minimizando una función de pérdida o coste (*loss* o *cost function*, en inglés) mediante el método del descenso por el gradiente, que en su forma genérica consiste en lo siguiente:

- Supongamos una función de coste $C(v_1, \dots, v_n)$.
- Si los argumentos cambian en una pequeña cantidad, $\Delta v_1, \dots, \Delta v_n$, entonces la teoría del cálculo infinitesimal nos dice que el valor de C cambia aproximadamente como

$$\begin{aligned}\Delta C &= C(v_1 + \Delta v_1, \dots, v_n + \Delta v_n) - C(v_1, \dots, v_n) \\ &\cong \frac{\partial C}{\partial v_1}(v_1, \dots, v_n)\Delta v_1 + \dots + \frac{\partial C}{\partial v_n}(v_1, \dots, v_n)\Delta v_n \\ &= \nabla C^T \Delta \mathbf{v}\end{aligned}\quad (*)$$

donde $\nabla C = \left(\frac{\partial C}{\partial v_1}(v_1, \dots, v_n), \dots, \frac{\partial C}{\partial v_n}(v_1, \dots, v_n) \right)^T$ es el gradiente de C en $(v_1, \dots, v_n)$ y $\Delta \mathbf{v} = (\Delta v_1, \dots, \Delta v_n)^T$.

- Una condición suficiente para conseguir que $\Delta C \leq 0$, es decir, que el valor de C disminuya (o, al menos, que no aumente) es elegir $\Delta \mathbf{v} = -\eta \nabla C$, donde $\eta > 0$ es el *factor de aprendizaje*.

En efecto, en ese caso se tiene

$$\Delta C \cong \nabla C^T \Delta \mathbf{v} = -\eta \nabla C^T \nabla C = -\eta \left(\left(\frac{\partial C}{\partial v_1}(v_1, \dots, v_n) \right)^2 + \dots + \left(\frac{\partial C}{\partial v_n}(v_1, \dots, v_n) \right)^2 \right)$$

De hecho, se puede demostrar que $-\nabla C$ proporciona la dirección en la que se obtiene el mayor decremento de C .

El método del descenso por el gradiente para un modelo de regresión realiza dos asunciones acerca de la función de coste:

- El coste $C_{(x,y)}$ para cada ejemplo únicamente depende de la salida del modelo para ese ejemplo y , por tanto, se puede entender como una función de los parámetros del modelo.
- El coste total se calcula como el promedio de los costes para cada ejemplo del conjunto de entrenamiento: $C = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} C_{(x,y)}$.

El algoritmo para entrenar una neurona artificial mediante descenso por el gradiente es, entonces, el siguiente:

- 1 Inicializar los pesos aleatoriamente
- 2 **Repetir** hasta que se cumpla la condición de terminación
- 3 Actualizar los pesos en la dirección $-\nabla C$
- 4 **Devolver** los pesos

donde cada peso w_i se actualiza así:

$$w_i \leftarrow w_i - \eta \frac{\partial C}{\partial w_i}$$

Es decir,

$$w_i \leftarrow w_i - \frac{\eta}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \frac{\partial C_{(x,y)}}{\partial w_i}$$

Por ejemplo, en el caso de que la función de coste considerada fuera el error cuadrático medio

$$\text{MSE} = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} (y - a_x)^2$$

donde $\mathbf{w}$ son los pesos, g la función de activación y $a_x = g(\mathbf{w}^T \mathbf{x})$ la salida de la neurona para $\mathbf{x}$, entonces

$$w_i \leftarrow w_i - \frac{2\eta}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} (y - a_x)(-g'(\mathbf{w}^T \mathbf{x}))x_i$$

A la hora de aplicar el algoritmo para entrenar una neurona artificial debe tenerse en cuenta lo siguiente:

- La función de activación de la neurona artificial debe ser diferenciable.
- El factor de aprendizaje debe ser lo suficientemente pequeño para que se cumpla la aproximación (*), pero suficientemente grande para que el algoritmo no minimice la función de coste con demasiada lentitud.

Ejemplo: Entrenamiento de una neurona artificial

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Supongamos la función sigmoide σ como función de activación y MSE como función de coste. En este caso, la salida de la neurona para un vector de argumentos $\mathbf{x}$ es $a_x = \sigma(\mathbf{w}^T \mathbf{x})$ y, por las propiedades de σ , se tiene que $\sigma'(\mathbf{w}^T \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})(1 - \sigma(\mathbf{w}^T \mathbf{x})) = a_x(1 - a_x)$. Por lo tanto, la actualización de los pesos es

$$w_i \leftarrow w_i - \frac{2\eta}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} (a_x - y)a_x(1 - a_x)x_i$$

Tomando $w_0 = 0.1$, $w_1 = -0.1$ y $w_2 = 0.1$ como pesos iniciales y $\eta = 0.1$ como factor de aprendizaje, la primera actualización de los pesos sería:

- Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 = 0.1$, por lo que la salida del perceptrón es $a_{(0,0)} = \sigma(z) \approx 0.525$.
- Para $x_1 = 0$ y $x_2 = 1$ se tiene que $z = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 = 0.2$, por lo que la salida del perceptrón es $a_{(0,1)} = \sigma(z) \approx 0.55$.
- Para $x_1 = 1$ y $x_2 = 0$ se tiene que $z = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 = 0$, por lo que la salida del perceptrón es $a_{(1,0)} = \sigma(z) = 0.5$.

- Para $x_1 = 1$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.1$, por lo que la salida del perceptrón es $a_{(1,1)} = \sigma(z) \cong 0.525$.
- El peso w_0 se actualiza como sigue:

$$w_0 \leftarrow 0.1 - \frac{0.2}{4}((0.525 - 0)0.525(1 - 0.525)1 + (0.55 - 1)0.55(1 - 0.55)1 + (0.5 - 1)0.5(1 - 0.5)1 + (0.525 - 1)0.525(1 - 0.525)1) \\ \cong 0.111$$

- El peso w_1 se actualiza como sigue:

$$w_1 \leftarrow -0.1 - \frac{0.2}{4}((0.525 - 0)0.525(1 - 0.525)0 + (0.55 - 1)0.55(1 - 0.55)0 + (0.5 - 1)0.5(1 - 0.5)1 + (0.525 - 1)0.525(1 - 0.525)1) \\ \cong -0.088$$

- El peso w_2 se actualiza como sigue:

$$w_2 \leftarrow 0.1 - \frac{0.2}{4}((0.525 - 0)0.525(1 - 0.525)0 + (0.55 - 1)0.55(1 - 0.55)1 + (0.5 - 1)0.5(1 - 0.5)0 + (0.525 - 1)0.525(1 - 0.525)1) \\ \cong 0.111$$

El algoritmo continuaría actualizando los pesos de la misma manera, hasta que se cumpla la condición de terminación como pueden ser, por ejemplo, que se haya realizado un cierto número de actualizaciones de los pesos o que el valor de la función de coste se halle por debajo de un cierto umbral.

Nótese como, en nuestro ejemplo, el error cuadrático medio cometido por la neurona artificial se ha reducido.

- Con los valores iniciales de los pesos, el error es

$$\text{MSE} \cong \frac{1}{4}((0 - 0.525)^2 + (1 - 0.55)^2 + (1 - 0.5)^2 + (1 - 0.525)^2) \cong 0.238$$

- Tras la primera actualización de los pesos,
 - Para $x_1 = 0$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.111$, por lo que la salida del perceptrón es $a_{(0,0)} = \sigma(z) \cong 0.528$.

- Para $x_1 = 0$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.222$, por lo que la salida del perceptrón es $a_{(0,1)} = \sigma(z) \approx 0.555$.
- Para $x_1 = 1$ y $x_2 = 0$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.023$, por lo que la salida del perceptrón es $a_{(1,0)} = \sigma(z) \approx 0.506$.
- Para $x_1 = 1$ y $x_2 = 1$ se tiene que $z = w_0 + w_1x_1 + w_2x_2 = 0.134$, por lo que la salida del perceptrón es $a_{(1,1)} = \sigma(z) \approx 0.533$.
- El error es

$$\text{MSE} \approx \frac{1}{4}((0 - 0.528)^2 + (1 - 0.555)^2 + (1 - 0.506)^2 + (1 - 0.533)^2) \approx 0.235$$

Algoritmo de retropropagación

El algoritmo de descenso por el gradiente se usa también para entrenar las redes neuronales genéricas. En este caso, una aplicación directa de la regla de la cadena para calcular las derivadas parciales de la función de coste respecto a los pesos sería computacionalmente ineficiente. El algoritmo de retropropagación (*backpropagation*, en inglés) calcula estas derivadas de manera eficiente, evitando repetir cálculos.

Las siguientes notaciones permitirán simplificar la descripción del algoritmo:

- L es la cantidad de capas de la red neuronal, a las cuales numeramos de 1 a L .
- n^l es la cantidad de neuronas en la capa l , a las cuales numeramos de 1 a n^l .
- $w_{j \leftarrow k}^l$ es el peso de la conexión entre la neurona k en la capa $l - 1$ y la neurona j en la capa l .
- z_j^l y a_j^l son la entrada y la salida, respectivamente, de la neurona j en la capa l .
- Consideramos que cada capa tiene una neurona 0 con $a_0^l = 1$. Entonces $w_{j \leftarrow 0}^l$ es el sesgo de la neurona j en la capa l .
- Dados dos vectores de la misma dimensión, $\mathbf{v}^1 = (v_1^1, \dots, v_n^1)^T$ y $\mathbf{v}^2 = (v_1^2, \dots, v_n^2)^T$, el producto de Hadamard es $\mathbf{v}^1 \odot \mathbf{v}^2 = (v_1^1 v_1^2, \dots, v_n^1 v_n^2)^T$.

Si g_j^l es la función de activación de la neurona j en la capa l , entonces

$$z_j^l = \sum_{k=1}^{n^{l-1}} w_{j \leftarrow k}^l a_k^{l-1} + w_{j \leftarrow 0}^l \quad \text{y} \quad a_j^l = g_j^l(z_j^l)$$

Es habitual que todas las neuronas en una misma capa usen la misma función de activación g^l . Podemos entonces calcular las salidas de las neuronas mediante operaciones matriciales:

$$\mathbf{z}^l = \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{w}_0^l \quad \text{y} \quad \mathbf{a}^l = g^l(\mathbf{z}^l)$$

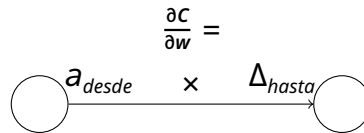
donde $\mathbf{z}^l = (z_1^l, \dots, z_{n_l}^l)^\top$ es el vector de entradas de la capa l , $\mathbf{W}^l = \begin{pmatrix} w_{1 \leftarrow 1}^l & \dots & w_{1 \leftarrow n^{l-1}}^l \\ \vdots & & \vdots \\ w_{n^l \leftarrow 1}^l & \dots & w_{n^l \leftarrow n^{l-1}}^l \end{pmatrix}$ la matriz de pesos, $\mathbf{a}^l = (a_1^l, \dots, a_{n_l}^l)^\top$ el vector de salidas, $\mathbf{w}_0^l = (w_{1 \leftarrow 0}^l, \dots, w_{n^l \leftarrow 0}^l)^\top$ el vector de sesgos y g^l la consideramos una función vectorizada (es decir, que aplicada a un vector se aplica elemento a elemento de ese vector).

El error Δ_j^l de la neurona j en la capa l es $\Delta_j^l \stackrel{\text{def}}{=} \frac{\partial C}{\partial z_j^l}$. Aplicando convenientemente la regla de la cadena se llega a

$$\begin{aligned}\Delta^L &= \nabla_{\mathbf{a}^L} C \odot (g^L)'(\mathbf{z}^L) \\ \Delta^l &= ((\mathbf{W}^{l+1})^\top \Delta^{l+1}) \odot (g^l)'(\mathbf{z}^l), \quad \text{para } l = L-1, \dots, 2\end{aligned}$$

donde $\Delta^l = \left(\frac{\partial C}{\partial z_1^l}, \dots, \frac{\partial C}{\partial z_{n_l}^l} \right)^\top$ y $\nabla_{\mathbf{a}^L} C = \left(\frac{\partial C}{\partial a_1^L}, \dots, \frac{\partial C}{\partial a_{n_L}^L} \right)^\top$.

De nuevo aplicando la regla de la cadena se llega a $\frac{\partial C}{\partial w_{j \leftarrow k}^l} = a_k^{l-1} \Delta_j^l$. Es decir, la componente de ∇C correspondiente al peso de una conexión entre dos neuronas es el producto de la salida de la neurona de partida por el error de la neurona de llegada, lo que gráficamente se puede representar como



Estamos ya en condiciones de plantear el algoritmo de retropropagación.

- 1 Inicializar los pesos aleatoriamente
- 2 **Repetir** hasta que se cumpla la condición de terminación
- 3 **Para cada** ejemplo $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}$ **hacer**
- 4 $\mathbf{a}^1 \leftarrow \mathbf{x}$
- 5 (Propagar salidas hacia delante) **Para cada** $l = 2, \dots, L$ **hacer**
- 6 $\mathbf{z}^l \leftarrow \mathbf{W}^l \mathbf{a}^{l-1} + \mathbf{w}_0^l$ y $\mathbf{a}^l \leftarrow g^l(\mathbf{z}^l)$
- 7 $\Delta^L \leftarrow \nabla_{\mathbf{a}^L} C \odot (g^L)'(\mathbf{z}^L)$
- 8 (Propagar errores hacia atrás) **Para cada** $l = L-1, \dots, 2$ **hacer**
- 9 $\Delta^l \leftarrow ((\mathbf{W}^{l+1})^\top \Delta^{l+1}) \odot (g^l)'(\mathbf{z}^l)$
- 10 **Para cada** peso $w_{j \leftarrow k}^l$ **hacer**
- 11 $\frac{\partial C(x,y)}{\partial w_{j \leftarrow k}^l} \leftarrow a_k^{l-1} \Delta_j^l$
- 12 **Para cada** peso $w_{j \leftarrow k}^l$ **hacer**
- 13 $w_{j \leftarrow k}^l \leftarrow w_{j \leftarrow k}^l - \frac{\eta}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \frac{\partial C(x,y)}{\partial w_{j \leftarrow k}^l}$
- 14 **Devolver** los pesos

Actualizar los pesos promediando el gradiente de C sobre todos los ejemplos de entrenamiento es costoso. El método del descenso estocástico por el gradiente (*stochastic gradient descent*, en inglés) aproxima el promedio a partir de un pequeño minilote (*minibatch*, en inglés) de ejemplos de entrenamiento elegidos aleatoriamente (es decir,

un suconjunto aleatorio del lote completo de ejemplos de entrenamiento). Cuando en el bucle del algoritmo se han considerado todos los ejemplos se dice que ha transcurrido una época (*epoch*, en inglés).

La elección de la capa de salida, de la función de activación en esa capa y de la función de coste a minimizar para entrenar la red neuronal es dependiente del problema que se pretenda resolver. Habitualmente se suele realizar lo siguiente:

- Para una tarea de regresión multisalida, es decir, en la que hay que predecir el valor de n atributos continuos, con $n \geq 1$:

- Se considera una capa de salida con $n^L = n$ neuronas artificiales.
- Se establece como función de activación g^L para la capa de salida la función identidad. La salida de la red es, por lo tanto, $\mathbf{a}^L = g^L(\mathbf{z}^L) = \mathbf{W}^L \mathbf{a}^{L-1} + \mathbf{w}_0^L$ y la tarea se resuelve simplemente proporcionando esa tupla de valores.
- Se considera como función de coste el error cuadrático medio:

$$\frac{1}{n} \sum_{k=1}^n (y_k - a_k^L)^2$$

Nótese que, puesto que se compara la salida de la red con la respuesta correcta y ambas son una tupla de n valores, deben agregarse los errores cometidos por cada neurona de salida. En este caso se ha elegido realizar esa agregación mediante una media aritmética, aunque otra opción que se suele utilizar es sumar esos errores individuales.

- El gradiente de la función de coste respecto de la salida de la red y el vector de errores en la capa de salida son, entonces,

$$\begin{aligned} \nabla_{\mathbf{a}^L} C &= \frac{2}{n} (\mathbf{a}^L - \mathbf{y}) \\ \Delta^L &= \nabla_{\mathbf{a}^L} C \odot \mathbf{1} = \frac{2}{n} (\mathbf{a}^L - \mathbf{y}) \end{aligned}$$

- Para una tarea de clasificación binaria:

- Las dos posibles clases se codifican con 0 y 1.
- Se considera una capa de salida con $n^L = 1$ neurona artificial.
- Se establece como función de activación g^L para la capa de salida la función sigmoide. La salida de la red es, por lo tanto, $a^L = g^L(z^L) = \sigma(\mathbf{W}^L \mathbf{a}^{L-1} + w_0^L)$ y la tarea se resuelve prefijando un umbral u y proporcionando la clase 1 si a^L supera ese umbral y 0 en caso contrario.
- Se considera como función de coste la entropía cruzada binaria:

$$-y \log_e(a^L) - (1 - y) \log_e(1 - a^L)$$

- El gradiente de la función de coste respecto de la salida de la red y el error en la capa de salida son, entonces,

$$\begin{aligned} \nabla_{a^L} C &= -\frac{y}{a^L} + \frac{1 - y}{1 - a^L} = \frac{a^L - y}{a^L(1 - a^L)} \\ \Delta^L &= \nabla_{a^L} C \times a^L \times (1 - a^L) = a^L - y \end{aligned}$$

■ Para una tarea de clasificación multiclase:

- Se realiza una codificación *one-hot* de las n posibles clases.
- Se considera una capa de salida con $n^L = n$ neuronas artificiales.
- Se establece como función de activación g^L para la capa de salida la función

$$\text{softmax}(z_1, \dots, z_n) = \left(\frac{e^{z_1}}{\sum_{i=1}^n e^{z_i}}, \dots, \frac{e^{z_n}}{\sum_{i=1}^n e^{z_i}} \right)$$

La salida de la red es, por lo tanto, $\mathbf{a}^L = g^L(\mathbf{z}^L) = \text{softmax}(\mathbf{W}^L \mathbf{a}^{L-1} + \mathbf{w}_0^L)$ y la tarea se resuelve proporcionando la clase $\arg \max_{k=1, \dots, n} (a_k^L)$.

- Se considera como función de coste la entropía cruzada categórica:

$$-\sum_{k=1}^n y_k \log_e(a_k^L)$$

- El gradiente de la función de coste respecto de la salida de la red y el vector de errores en la capa de salida son, entonces,

$$\nabla_{\mathbf{a}^L} C = -\frac{\mathbf{y}}{\mathbf{a}^L}$$

$$\Delta^L = \begin{pmatrix} a_1^L(1-a_1^L) & -a_2^L a_1^L & \dots & -a_n^L a_1^L \\ \dots & \dots & \dots & \dots \\ -a_1^L a_n^L & -a_2^L a_n^L & \dots & a_n^L(1-a_n^L) \end{pmatrix} \times \nabla_{\mathbf{a}^L} C = \mathbf{a}^L - \mathbf{y}$$

La regla de la cadena establece, de forma general, que

$$\Delta_j^L = \frac{\partial C}{\partial z_j^L} = \sum_{k=1}^n \frac{\partial C}{\partial a_k^L} \frac{\partial a_k^L}{\partial z_j^L}$$

Es decir,

$$\Delta^L = \begin{pmatrix} \frac{\partial a_1^L}{\partial z_1^L} & \dots & \frac{\partial a_n^L}{\partial z_1^L} \\ \dots & \dots & \dots \\ \frac{\partial a_1^L}{\partial z_n^L} & \dots & \frac{\partial a_n^L}{\partial z_n^L} \end{pmatrix} \times \nabla_{\mathbf{a}^L} C$$

En los dos primeros casos, en los que la función de activación g es la función de identidad y la función sigmoide, la salida de cada neurona de la capa de salida solo depende de su propia entrada: $a_1^L = g(z_1^L), \dots, a_n^L = g(z_n^L)$. Por lo tanto,

$$\Delta^L = \begin{pmatrix} \frac{\partial g(z_1^L)}{\partial z_1^L} & 0 & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & \dots & 0 & \frac{\partial g(z_n^L)}{\partial z_n^L} \end{pmatrix} \times \nabla_{\mathbf{a}^L} C = \nabla_{\mathbf{a}^L} C \odot g'(\mathbf{z}^L)$$

Sin embargo, en el tercer caso en el que la función de activación g es la función *softmax*, la salida de cada neurona de la capa de salida depende de las entradas de todas las neuronas de esa capa: $a_1^L = g(z_1^L, \dots, z_n^L), \dots, a_n^L = g(z_1^L, \dots, z_n^L)$. La expresión, por tanto, no se puede simplificar.

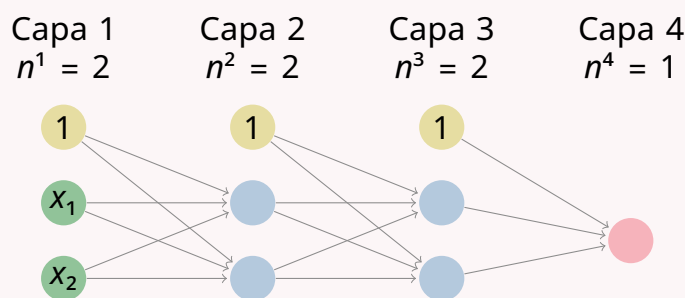
Las funciones de coste anteriores se derivan de asumir que la salida de la red modeliza una distribución de probabilidad sobre los atributos objetivos (una distribución normal multidimensional $N(\mathbf{a}^L, \mathbf{1})$, una distribución de Bernoulli con probabilidad de éxito a^L y una distribución categórica con vector $\mathbf{a}^L$ de probabilidades, respectivamente) y considerar una estimación de máxima verosimilitud del conjunto de entrenamiento, calculada de una forma numéricamente estable.

Ejemplo: Descenso estocástico por el gradiente durante una época

Consideremos, para una tarea de clasificación binaria, el conjunto de entrenamiento $\mathcal{D}$

	Entrada: $\mathbf{x} = (x_1, x_2)^T$	Objetivo: y
E_1	$(0.78, -0.67)^T$	0
E_2	$(-0.73, -1.38)^T$	0
E_3	$(-0.64, 0.88)^T$	1
E_4	$(-0.80, 0.48)^T$	1

y la red neuronal



La función de activación de las neuronas artificiales es la función rectificador en las capas ocultas y la función sigmoide en la capa de salida. Las matrices iniciales de pesos y sesgos son

$$\mathbf{W}^2 = \begin{pmatrix} -0.67 & -0.74 \\ -0.10 & 0.23 \end{pmatrix}$$

$$\mathbf{w}_0^2 = \begin{pmatrix} 0.68 \\ 1.11 \end{pmatrix}$$

$$\mathbf{W}^3 = \begin{pmatrix} 0.73 & -1.78 \\ 0.09 & 0.56 \end{pmatrix}$$

$$\mathbf{w}_0^3 = \begin{pmatrix} 0.17 \\ -0.29 \end{pmatrix}$$

$$\mathbf{W}^4 = \begin{pmatrix} 0.91 & -0.89 \end{pmatrix}$$

$$\mathbf{w}_0^4 = (0.65)$$

Veamos cómo se entrenaría la red neuronal durante una época mediante el método del descenso estocástico por el gradiente asumiendo que los ejemplos del conjunto de entrenamiento se han distribuido aleatoriamente en los siguientes dos minilotes:

$$B_1 = \{E_2, E_3\} \quad B_2 = \{E_1, E_4\}$$

Consideramos $\eta = 0.1$ como factor de aprendizaje y la entropía cruzada binaria como función de coste a minimizar.

Nota: los resultados de los cálculos se han redondeado a cuatro decimales.

■ Minilote B_1 :

- Ejemplo E_2 : $\mathbf{x} = \begin{pmatrix} -0.73 \\ -1.38 \end{pmatrix}$, $y = 0$

$$\mathbf{a}^1 = \mathbf{x} = \begin{pmatrix} -0.73 \\ -1.38 \end{pmatrix}$$

$$\mathbf{z}^2 = \mathbf{W}^2 \mathbf{a}^1 + \mathbf{w}_0^2 \cong \begin{pmatrix} 2.1903 \\ 0.8656 \end{pmatrix} \quad \mathbf{a}^2 = \text{rectificador}(\mathbf{z}^2) \cong \begin{pmatrix} 2.1903 \\ 0.8656 \end{pmatrix}$$

$$\mathbf{z}^3 = \mathbf{W}^3 \mathbf{a}^2 + \mathbf{w}_0^3 \cong \begin{pmatrix} 0.2282 \\ 0.3919 \end{pmatrix} \quad \mathbf{a}^3 = \text{rectificador}(\mathbf{z}^3) \cong \begin{pmatrix} 0.2282 \\ 0.3919 \end{pmatrix}$$

$$\mathbf{z}^4 = \mathbf{W}^4 \mathbf{a}^3 + \mathbf{w}_0^4 \cong (0.5089) \quad \mathbf{a}^4 = \sigma(\mathbf{z}^4) \cong (0.6245)$$

$$\Delta^4 = \mathbf{a}^4 - y = (0.6245)$$

$$\Delta^3 = ((\mathbf{W}^4)^T \Delta^4) \odot \text{rectificador}'(\mathbf{z}^3) \cong \begin{pmatrix} 0.5683 \\ -0.5558 \end{pmatrix}$$

$$\Delta^2 = ((\mathbf{W}^3)^T \Delta^3) \odot \text{rectificador}'(\mathbf{z}^2) \cong \begin{pmatrix} 0.3648 \\ -1.3228 \end{pmatrix}$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 0}^2} = 1 \Delta_1^2 \cong 0.3648$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 1}^2} = a_1^1 \Delta_1^2 \cong -0.2663$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 2}^2} = a_2^1 \Delta_1^2 \cong -0.5034$$

$$\frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 0}^2} = 1 \Delta_2^2 \cong -1.3228$$

$$\frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 1}^2} = a_1^1 \Delta_2^2 \cong 0.9656$$

$$\frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 2}^2} = a_2^1 \Delta_2^2 \cong 1.8255$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 0}^3} = 1 \Delta_1^3 \cong 0.5683$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 1}^3} = a_1^2 \Delta_1^3 \cong 1.2447$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 2}^3} = a_2^2 \Delta_1^3 \cong 0.4919$$

$$\frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 0}^3} = 1 \Delta_2^3 \cong -0.5558$$

$$\frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 1}^3} = a_1^2 \Delta_2^3 \cong -1.2174$$

$$\frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 2}^3} = a_2^2 \Delta_2^3 \cong -0.4811$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 0}^4} = 1 \Delta_1^4 \cong 0.6245$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 1}^4} = a_1^3 \Delta_1^4 \cong 0.1425$$

$$\frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 2}^4} = a_2^3 \Delta_1^4 \cong 0.2447$$

- Ejemplo E_3 : $\mathbf{x} = \begin{pmatrix} -0.64 \\ 0.88 \end{pmatrix}$, $y = 1$

$$\mathbf{a}^1 = \mathbf{x} = \begin{pmatrix} -0.64 \\ 0.88 \end{pmatrix}$$

$$\mathbf{z}^2 = \mathbf{W}^2 \mathbf{a}^1 + \mathbf{w}_0^2 \cong \begin{pmatrix} 0.4576 \\ 1.3764 \end{pmatrix} \quad \mathbf{a}^2 = \text{rectificador}(\mathbf{z}^2) \cong \begin{pmatrix} 0.4576 \\ 1.3764 \end{pmatrix}$$

$$\mathbf{z}^3 = \mathbf{W}^3 \mathbf{a}^2 + \mathbf{w}_0^3 \cong \begin{pmatrix} -1.9459 \\ 0.5520 \end{pmatrix} \quad \mathbf{a}^3 = \text{rectificador}(\mathbf{z}^3) \cong \begin{pmatrix} 0 \\ 0.5520 \end{pmatrix}$$

$$\mathbf{z}^4 = \mathbf{W}^4 \mathbf{a}^3 + \mathbf{w}_0^4 \cong (0.1854) \quad \mathbf{a}^4 = \sigma(\mathbf{z}^4) \cong (0.5462)$$

$$\Delta^4 = \mathbf{a}^4 - y = (-0.4538)$$

$$\Delta^3 = ((\mathbf{W}^4)^T \Delta^4) \odot \text{rectificador}'(\mathbf{z}^3) \cong \begin{pmatrix} 0 \\ 0.4039 \end{pmatrix}$$

$$\Delta^2 = ((\mathbf{W}^3)^T \Delta^3) \odot \text{rectificador}'(\mathbf{z}^2) \cong \begin{pmatrix} 0.0364 \\ 0.2262 \end{pmatrix}$$

$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 0}^2} = 1\Delta_1^2 \cong 0.0364$	$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 1}^2} = a_1^1 \Delta_1^2 \cong -0.0233$	$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 2}^2} = a_2^1 \Delta_1^2 \cong 0.0320$
$\frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 0}^2} = 1\Delta_2^2 \cong 0.2262$	$\frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 1}^2} = a_1^2 \Delta_2^2 \cong -0.1448$	$\frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 2}^2} = a_2^2 \Delta_2^2 \cong 0.1991$
$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 0}^3} = 1\Delta_1^3 \cong 0$	$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 1}^3} = a_1^2 \Delta_1^3 \cong 0$	$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 2}^3} = a_2^2 \Delta_1^3 \cong 0$
$\frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 0}^3} = 1\Delta_2^3 \cong 0.4039$	$\frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 1}^3} = a_1^2 \Delta_2^3 \cong 0.1848$	$\frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 2}^3} = a_2^2 \Delta_2^3 \cong 0.5559$
$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 0}^4} = 1\Delta_1^4 \cong -0.4538$	$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 1}^4} = a_1^3 \Delta_1^4 \cong 0$	$\frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 2}^4} = a_2^3 \Delta_1^4 \cong -0.2369$

- Actualización de los pesos:

$$\mathbf{W}^2 = \mathbf{W}^2 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 1}^2} & \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 2}^2} \\ \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 1}^2} & \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 2}^2} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 1}^2} & \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 2}^2} \\ \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 1}^2} & \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 2}^2} \end{pmatrix} \right) \cong \begin{pmatrix} -0.6555 & -0.7164 \\ -0.1410 & 0.1288 \end{pmatrix}$$

$$\mathbf{w}_0^2 = \mathbf{w}_0^2 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 0}^2} \\ \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 0}^2} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 0}^2} \\ \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 0}^2} \end{pmatrix} \right) \cong \begin{pmatrix} 0.6599 \\ 1.1648 \end{pmatrix}$$

$$\mathbf{W}^3 = \mathbf{W}^3 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 1}^3} & \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 2}^3} \\ \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 1}^3} & \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 2}^3} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 1}^3} & \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 2}^3} \\ \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 1}^3} & \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 2}^3} \end{pmatrix} \right) \cong \begin{pmatrix} 0.6678 & -1.8046 \\ 0.1416 & 0.5563 \end{pmatrix}$$

$$\mathbf{w}_0^3 = \mathbf{w}_0^3 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 0}^3} \\ \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 0}^3} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 0}^3} \\ \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 0}^3} \end{pmatrix} \right) \cong \begin{pmatrix} 0.1416 \\ -0.2824 \end{pmatrix}$$

$$\mathbf{W}^4 = \mathbf{W}^4 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 1}^4} & \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 2}^4} \\ \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 1}^4} & \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 2}^4} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 1}^4} & \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 2}^4} \\ \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 1}^4} & \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 2}^4} \end{pmatrix} \right) \cong \begin{pmatrix} 0.9029 & -0.8904 \end{pmatrix}$$

$$\mathbf{w}_0^4 = \mathbf{w}_0^4 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_2}}{\partial w_{1 \leftarrow 0}^4} \\ \frac{\partial C_{E_2}}{\partial w_{2 \leftarrow 0}^4} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_3}}{\partial w_{1 \leftarrow 0}^4} \\ \frac{\partial C_{E_3}}{\partial w_{2 \leftarrow 0}^4} \end{pmatrix} \right) \cong \begin{pmatrix} 0.6415 \end{pmatrix}$$

■ Minilote B_2 :

- Ejemplo E_1 : $\mathbf{x} = \begin{pmatrix} 0.78 \\ -0.67 \end{pmatrix}$, $y = 0$

$$\mathbf{a}^1 = \mathbf{x} = \begin{pmatrix} 0.78 \\ -0.67 \end{pmatrix}$$

$$\mathbf{z}^2 = \mathbf{W}^2 \mathbf{a}^1 + \mathbf{w}_0^2 \cong \begin{pmatrix} 0.6286 \\ 0.9685 \end{pmatrix} \quad \mathbf{a}^2 = \text{rectificador}(\mathbf{z}^2) \cong \begin{pmatrix} 0.6286 \\ 0.9685 \end{pmatrix}$$

$$\mathbf{z}^3 = \mathbf{W}^3 \mathbf{a}^2 + \mathbf{w}_0^3 \cong \begin{pmatrix} -1.1864 \\ 0.3454 \end{pmatrix} \quad \mathbf{a}^3 = \text{rectificador}(\mathbf{z}^3) \cong \begin{pmatrix} 0 \\ 0.3454 \end{pmatrix}$$

$$\mathbf{z}^4 = \mathbf{W}^4 \mathbf{a}^3 + \mathbf{w}_0^4 \cong (0.3340) \quad \mathbf{a}^4 = \sigma(\mathbf{z}^4) \cong (0.5827)$$

$$\Delta^4 = \mathbf{a}^4 - y = (0.5827)$$

$$\Delta^3 = ((\mathbf{W}^4)^T \Delta^4) \odot \text{rectificador}'(\mathbf{z}^3) \cong \begin{pmatrix} 0 \\ -0.5188 \end{pmatrix}$$

$$\Delta^2 = ((\mathbf{W}^3)^T \Delta^3) \odot \text{rectificador}'(\mathbf{z}^2) \cong \begin{pmatrix} -0.0735 \\ -0.2886 \end{pmatrix}$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 0}^2} = 1 \Delta_1^2 \cong -0.0735$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 1}^2} = a_1^1 \Delta_1^2 \cong -0.0573$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 2}^2} = a_2^1 \Delta_1^2 \cong 0.0492$$

$$\frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 0}^2} = 1 \Delta_2^2 \cong -0.2886$$

$$\frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 1}^2} = a_1^1 \Delta_2^2 \cong -0.2251$$

$$\frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 2}^2} = a_2^1 \Delta_2^2 \cong 0.1934$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 0}^3} = 1 \Delta_1^3 \cong 0$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 1}^3} = a_1^2 \Delta_1^3 \cong 0$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 2}^3} = a_2^2 \Delta_1^3 \cong 0$$

$$\frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 0}^3} = 1 \Delta_2^3 \cong -0.5188$$

$$\frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 1}^3} = a_1^2 \Delta_2^3 \cong -0.3261$$

$$\frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 2}^3} = a_2^2 \Delta_2^3 \cong -0.5025$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 0}^4} = 1 \Delta_1^4 \cong 0.5827$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 1}^4} = a_1^3 \Delta_1^4 \cong 0$$

$$\frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 2}^4} = a_2^3 \Delta_1^4 \cong 0.2013$$

- Ejemplo E_4 : $\mathbf{x} = \begin{pmatrix} -0.80 \\ 0.48 \end{pmatrix}$, $y = 1$

$$\mathbf{a}^1 = \mathbf{x} = \begin{pmatrix} -0.80 \\ 0.48 \end{pmatrix}$$

$$\mathbf{z}^2 = \mathbf{W}^2 \mathbf{a}^1 + \mathbf{w}_0^2 \cong \begin{pmatrix} 0.8404 \\ 1.3394 \end{pmatrix} \quad \mathbf{a}^2 = \text{rectificador}(\mathbf{z}^2) \cong \begin{pmatrix} 0.8404 \\ 1.3394 \end{pmatrix}$$

$$\mathbf{z}^3 = \mathbf{W}^3 \mathbf{a}^2 + \mathbf{w}_0^3 \cong \begin{pmatrix} -1.7143 \\ 0.5817 \end{pmatrix} \quad \mathbf{a}^3 = \text{rectificador}(\mathbf{z}^3) \cong \begin{pmatrix} 0 \\ 0.5817 \end{pmatrix}$$

$$\mathbf{z}^4 = \mathbf{W}^4 \mathbf{a}^3 + \mathbf{w}_0^4 \cong (0.1236) \quad \mathbf{a}^4 = \sigma(\mathbf{z}^4) \cong (0.5309)$$

$$\Delta^4 = \mathbf{a}^4 - y = (-0.4691)$$

$$\Delta^3 = ((\mathbf{W}^4)^T \Delta^4) \odot \text{rectificador}'(\mathbf{z}^3) \cong \begin{pmatrix} 0 \\ 0.4177 \end{pmatrix}$$

$$\Delta^2 = ((\mathbf{W}^3)^T \Delta^3) \odot \text{rectificador}'(\mathbf{z}^2) \cong \begin{pmatrix} 0.0591 \\ 0.2324 \end{pmatrix}$$

$$\begin{array}{lll} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 0}^2} = 1\Delta_1^2 \cong 0.0591 & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 1}^2} = a_1^1 \Delta_1^2 \cong -0.0473 & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 2}^2} = a_2^1 \Delta_1^2 \cong 0.0284 \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 0}^2} = 1\Delta_2^2 \cong 0.2324 & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 1}^2} = a_1^2 \Delta_2^2 \cong -0.1859 & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 2}^2} = a_2^2 \Delta_2^2 \cong 0.1116 \\ \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 0}^3} = 1\Delta_1^3 \cong 0 & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 1}^3} = a_1^2 \Delta_1^3 \cong 0 & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 2}^3} = a_2^2 \Delta_1^3 \cong 0 \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 0}^3} = 1\Delta_2^3 \cong 0.4177 & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 1}^3} = a_1^2 \Delta_2^3 \cong 0.3510 & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 2}^3} = a_2^2 \Delta_2^3 \cong 0.5595 \\ \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 0}^4} = 1\Delta_1^4 \cong -0.4691 & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 1}^4} = a_1^3 \Delta_1^4 \cong 0 & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 2}^4} = a_2^3 \Delta_1^4 \cong -0.2729 \end{array}$$

- Actualización de los pesos:

$$\mathbf{W}^2 = \mathbf{W}^2 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 1}^2} & \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 2}^2} \\ \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 1}^2} & \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 2}^2} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 1}^2} & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 2}^2} \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 1}^2} & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 2}^2} \end{pmatrix} \right) \cong \begin{pmatrix} -0.6520 & -0.7194 \\ -0.1245 & 0.1133 \end{pmatrix}$$

$$\mathbf{w}_0^2 = \mathbf{w}_0^2 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 0}^2} \\ \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 0}^2} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 0}^2} \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 0}^2} \end{pmatrix} \right) \cong \begin{pmatrix} 0.6551 \\ 1.1419 \end{pmatrix}$$

$$\mathbf{W}^3 = \mathbf{W}^3 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 1}^3} & \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 2}^3} \\ \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 1}^3} & \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 2}^3} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 1}^3} & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 2}^3} \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 1}^3} & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 2}^3} \end{pmatrix} \right) \cong \begin{pmatrix} 0.6678 & -1.8046 \\ 0.1148 & 0.5005 \end{pmatrix}$$

$$\mathbf{w}_0^3 = \mathbf{w}_0^3 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 0}^3} \\ \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 0}^3} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 0}^3} \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 0}^3} \end{pmatrix} \right) \cong \begin{pmatrix} 0.1416 \\ -0.3235 \end{pmatrix}$$

$$\mathbf{W}^4 = \mathbf{W}^4 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 1}^4} & \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 2}^4} \\ \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 1}^4} & \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 2}^4} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 1}^4} & \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 2}^4} \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 1}^4} & \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 2}^4} \end{pmatrix} \right) \cong \begin{pmatrix} 0.9029 & -0.8649 \end{pmatrix}$$

$$\mathbf{w}_0^4 = \mathbf{w}_0^4 - \frac{\eta}{2} \left(\begin{pmatrix} \frac{\partial C_{E_1}}{\partial w_{1 \leftarrow 0}^4} \\ \frac{\partial C_{E_1}}{\partial w_{2 \leftarrow 0}^4} \end{pmatrix} + \begin{pmatrix} \frac{\partial C_{E_4}}{\partial w_{1 \leftarrow 0}^4} \\ \frac{\partial C_{E_4}}{\partial w_{2 \leftarrow 0}^4} \end{pmatrix} \right) \cong \begin{pmatrix} 0.6876 \end{pmatrix}$$

Se ha completado una época. Ahora se continuaría con la siguiente época, separando aleatoriamente el conjunto de entrenamiento en dos minilotes de tamaño 2 y actualizando de nuevo los pesos considerando primero los ejemplos del primer minilote y luego los del segundo minilote.